

Implémentez une recherche intelligente avec des requêtes en texte intégral, vectorielles et hybrides

Temps estimé : 45 minutes

Dans cet exercice, vous implémentez différentes approches de recherche dans Azure SQL Database. Vous créez des index en texte intégral, effectuez des recherches vectorielles en utilisant des embeddings stockés, et combinez les deux techniques avec la recherche hybride utilisant la fusion de rang réciproque (RRF). Vous comparez ensuite la manière dont chaque approche de recherche gère la même requête et observez les différences de résultats.

Vous êtes développeur de bases de données pour Adventure Works. Votre équipe souhaite améliorer la recherche de produits afin que les clients puissent trouver des articles pertinents, qu'ils recherchent par des mots-clés exacts ou qu'ils décrivent ce dont ils ont besoin en langage naturel. Vous implémentez la recherche en texte intégral pour la correspondance de mots-clés, la recherche vectorielle pour la similarité sémantique, et la recherche hybride pour combiner les deux approches.

! Ces exercices vous demandent de copier-coller du code T-SQL. Veuillez vérifier que le code a été copié correctement avant de l'exécuter.

Prérequis

- [Un abonnement Azure](#) avec approbation pour l'[accès Azure OpenAI](#)
- Visual Studio Code avec l'extension SQL Server (mssql), ou SQL Server Management Studio
- Connaissance de base de Azure SQL Database et T-SQL

Provision an Azure SQL Database

Tout d'abord, créez une base de données Azure SQL avec des données d'exemple.

! Passez cette section si vous avez déjà une base de données Azure SQL AdventureWorksLT provisionnée.

1. Allez sur le [hub Azure SQL](#) et connectez-vous avec votre compte Azure si vous le demandez. Dans le volet **Azure SQL Database**, sélectionnez **Afficher les options**, puis **Créer une base de données SQL**.

! Si vous voyez une bannière **d'offre gratuite** sur cette page, vous pouvez l'utiliser gratuitement pour utiliser Azure SQL Database. [L'offre gratuite](#) offre 100 000 vCore secondes de calcul serverless et 32 Go de stockage par mois. Si vous postulez pour l'offre gratuite, passez les étapes 3 à 6.

2. Dans l'onglet **Bases** de la page **Créer une base de données SQL**, remplissez les informations requises :

Cadre	Valeur
Abonnement	Sélectionnez votre abonnement Azure.
Groupe de ressources	Sélectionnez ou créez un groupe de ressources.
Nom de la base de données	<i>AdventureWorksLT</i>



Cadre

Valeur

Serveur

Sélectionnez **Créer un nouveau** serveur et créez un nouveau serveur avec un nom unique. Sélectionnez votre **localisation**. Pour l'authentification, sélectionnez l'une des options suivantes puis **sélectionnez OK** :

L'authentification

n'est pas optionnelle.

Vous devez choisir la méthode qui correspond aux politiques de sécurité de votre organisation. Chaque option influence la façon dont vous vous connectez à la base de données ultérieurement :

- **Utilisez l'authentification Microsoft Entra uniquement (recommandée)** : Sélectionnez ceci si votre organisation nécessite un accès basé sur Entra. Définissez votre compte Azure comme **administrateur Microsoft Entra**. Vous vous connectez à la base de données en utilisant votre compte Microsoft Entra (par exemple, dans SSMS, sélectionnez *Authentification* = **Microsoft Entra MFA**).
- **Utilisez à la fois l'authentification SQL et Microsoft Entra/SQL** : Sélectionnez ceci si vous préférez une connexion administrateur SQL ou si votre organisation autorise les deux méthodes. Fournissez un **identifiant administrateur** serveur et un **mot de passe**. Vous avez besoin de ces identifiants pour vous connecter. Vous pouvez aussi configurer un **administrateur Microsoft Entra** pour activer les connexions Entra en plus de l'auth-licence SQL.

La méthode d'authentification que vous choisissez ici contrôle la façon dont vous vous connectez à la base de données en tant que développeur. C'est distinct de la façon dont Azure SQL se connecte à Azure OpenAI, qui est configuré plus tard dans le laboratoire.

  Si vous avez déjà un serveur de test que vous pouvez utiliser, sélectionnez-le au lieu d'en créer un nouveau.

3. Laisser **Vouloir utiliser SQL elastic pool** réglé sur **Non**.
4. Pour **l'environnement de charge de travail**, sélectionnez **Développement**. Cela préconfigure le calcul en **mode serveur à usage général** avec l'auto-pause activée, qui est l'option payante la plus économique.
5. **Dans Calculer + stockage**, sélectionnez **Configurer la base de données**. Changez la couche de service en **Hyperscale** et la couche de calcul en **Serverless**. **Sélectionnez Postuler** pour confirmer.
6. **Dans la section Redondance du stockage de sauvegarde**, sélectionnez **Stockage de sauvegarde localement redondant**.
7. Sélectionner **Suivant : Réseautage**.
8. Dans l'onglet **Réseau**, pour **la méthode de connectivité**, sélectionnez **Terminaison public**.

  Si vous avez sélectionné un serveur existant au lieu d'en créer un nouveau, l'option **méthode de connectivité** peut ne pas apparaître car elle est déjà configurée sur le serveur.

9. Dans **les règles du pare-feu**, configurez **Oui : Autoriser les services et ressources Azure à accéder à ce serveur et Ajouter l'adresse IP client actuelle à Oui**.
10. Sélectionnez **Suivant : Sécurité**, puis sélectionnez **Suivant : Paramètres supplémentaires**.
11. Dans l'onglet **Paramètres supplémentaires**, sous **Source de données**, définissez **Utiliser les données existantes** sur *Échantillonnage* pour créer la base d'échantillons AdventureWorksLT. Sélectionnez **OK** quand on vous le demande pour confirmer.
12. Sélectionnez **Réviser + créer**, revoir les paramètres, puis **sélectionner Créer**.
13. Attendez la fin du déploiement, puis naviguez vers la nouvelle ressource Azure SQL Database.

Créez un projet Foundry et déployez des modèles Azure OpenAI

Ensuite, créez un projet dans Microsoft Foundry et déployez un modèle de chat et un modèle d'intégration.



!  Passez cette section si vous avez déjà une ressource Azure OpenAI avec des modèles **gpt-5.4-mini** et **text-embedding-3-small** déployés.

Créer un projet Foundry

La première étape consiste à créer un nouveau projet dans Microsoft Foundry, qui servira d'espace de travail pour déployer et gérer vos modèles Azure OpenAI.

1. Allez sur [Microsoft Foundry](#) et connectez-vous avec votre compte Azure.

!  Si vous voyez un bouton **de bascule New Foundry** dans le coin supérieur droit du portail, assurez-vous qu'il est **activé** pour utiliser la dernière version du portail Foundry. Les étapes ci-dessous supposent la nouvelle expérience.

2. Créez un nouveau projet :

- a. Si un nom de projet est affiché en haut à gauche, sélectionnez-le puis **sélectionnez Créer un nouveau projet**. Si aucun projet n'existe, sélectionnez **+ Créer projet** depuis la page d'accueil.
- b. Entrez un nom de projet (par exemple, *proj-sqlalab*).
- c. **Développez les options avancées**. Sélectionnez le même **groupe d'abonnement** et **de ressources** que vous avez utilisé pour votre base de données Azure SQL, puis sélectionnez une **région** où les modèles Azure OpenAI sont disponibles.
- d. **Sélectionne Créer**. Si une invite « **Allons-y** » apparaît, sélectionnez « **Allons-y** ».

Déployer Azure OpenAI models

Vous déployerez deux modèles : **gpt-5.4-mini** (pour les complétions de chat) et **text-embedding-3-small** (pour les embeddings). Les étapes suivantes vous guident dans le déploiement des deux modèles.

Déployons d'abord le modèle **GPT-5.4-mini**.

1. Sélectionnez **Découvrir** dans la barre de navigation supérieure. Dans la barre de recherche en haut de la page, cherchez **gpt-5.4-mini**. Sinon, sélectionnez **Afficher tous les modèles** dans la section **Modèles en vedette** pour parcourir le catalogue complet. Sélectionnez **gpt-5.4-mini** dans les résultats de recherche.
2. Sur la page des détails du modèle, sélectionnez le menu **déroulant Déploiement** et **choisissez Paramètres par défaut**.
3. Si une boîte de dialogue **Sélectionner un projet à déployer** apparaît, sélectionnez la même **région** que vous avez choisie lors de la création du projet, puis sélectionnez votre projet dans le menu **déroulant du projet**. **Sélectionnez Continuer**.
4. Définissez le **nom de déploiement** sur **gpt-5.4-mini** et sélectionnez **Déploiement**.
5. Une fois le déploiement terminé, le modèle est désormais déployé.

Déployons ensuite le modèle **text-embedding-3-small**.

1. Déploie maintenant le modèle d'intégration. Sélectionnez **Découvrir** dans la navigation supérieure et cherchez **text-embedding-3-small**. Sélectionnez-le dans les résultats de recherche.
2. Sur la page des détails du modèle, sélectionnez le menu **déroulant Déploiement** et **choisissez Paramètres par défaut**.
3. Si une boîte **de dialogue Sélectionner un projet à déployer** apparaît, sélectionnez la même **région** et **le même projet** qu'auparavant et **sélectionnez Continuer**.
4. Une fois le déploiement terminé, le modèle est désormais déployé.
5. **Sélectionnez Build > Models** dans la navigation supérieure pour vérifier que les deux déploiements apparaissent.

Récupérer le point de terminaison Azure OpenAI

Récupérez maintenant le **point de terminaison Azure OpenAI** dont vous avez besoin pour les étapes T-SQL plus tard.



1. Dans le [portail Azure](#), allez dans votre groupe de ressources et sélectionnez la ressource **Foundry** créée avec votre projet (par exemple, *proj-sqlailab-resource*).
2. Dans le menu de gauche, sélectionnez **Gestion des ressources** > **Clés et Terminaison**. Sélectionnez l'onglet **OpenAI** et notez l'URL du point de terminaison (par exemple,). Il te faudra plus tard le nom du terminau (la partie précédente). `https://proj-sqlailab-resource.openai.azure.com/` `.openai.azure.com`

! La page **Clés et Points d'Arrivée** comporte trois onglets : **Foundry**, **OpenAI** et **Services IA**. Assurez-vous de sélectionner l'onglet **OpenAI**. Cet onglet affiche le point d'extrémité dans le format requis par les bases de données Azure SQL et . L'onglet Foundry affiche une URL différente qui ne fonctionne pas avec ces fonctionnalités T-SQL.

`CREATE EXTERNAL MODEL` `sp_invoke_external_rest_endpoint` `.services.ai.azure.com`

Configurer l'accès à l'identité gérée

Puisque Azure SQL Database utilise une identité managée attribuée par le système pour s'authentifier avec Azure OpenAI, vous devez activer l'identité sur votre SQL Server et lui accorder l'accès à la ressource Azure OpenAI. Cette approche est plus sécurisée que les clés API et ne nécessite pas de stocker de secrets.

! Passez cette section si votre SQL Server dispose déjà d'une identité managée assignée par le système activée et qu'il a obtenu le rôle **Cognitive Services OpenAI User** sur votre ressource Azure OpenAI.

1. Dans le portail Azure, naviguez vers le **serveur SQL** que tu as créé plus tôt (le serveur logique, pas la base de données).
2. Dans le menu de gauche, sélectionnez **Sécurité** > **Identité**.
3. Dans la **section Système assigné à l'identité gérée**, réglez **Statut** sur **Activé** et sélectionnez **Enregistrer**.
4. Une fois l'identité gérée activée sur votre serveur SQL, naviguez vers votre ressource **Azure OpenAI** (par exemple, *adventureworks-openai*).
5. Dans le menu de gauche, sélectionnez **Contrôle d'accès (IAM)**.
6. Sélectionnez **+ Ajouter** puis sélectionnez **Ajouter l'attribution de rôle**.
7. Dans l'onglet **Rôle**, recherchez et sélectionnez **Utilisateur OpenAI des services cognitifs**, puis sélectionnez **Suivant**.
8. Dans l'onglet **Membres**, sélectionnez **Identité gérée**, puis sélectionnez **+ Sélectionnez membres**.
9. Dans le volet **Sélectionner les identités gérées**, définissez **l'identité managée** sur **serveur SQL**, sélectionnez votre serveur SQL dans la liste, puis sélectionnez **Sélectionner**.
10. **Sélectionnez Réviser + assigner** deux fois pour compléter l'attribution du rôle.

! L'attribution du rôle peut prendre jusqu'à 5 minutes avant de prendre effet. Vous pouvez procéder aux étapes suivantes en attendant.

Créer le tableau ProductReview

La base de données d'exemples AdventureWorksLT contient des informations sur les produits mais aucun avis client. À cette étape, vous téléchargez et exécutez un script qui crée un tableau **ProductReview** avec 140 avis réalistes répartis dans de nombreuses catégories de produits. Ces revues fournissent des données en texte enrichi pour la recherche en texte intégral, vectoriel et hybride.

1. Connectez-vous à votre base de données SQL Azure via Visual Studio Code (avec l'extension SQL Server) ou SQL Server Management Studio.



!  **La manière de se connecter** dépend de la méthode d'authentification prise en charge par votre organisation et qui a été configurée lors de la création du serveur :

- o **Authentification Microsoft Entra** : Dans SSMS, réglez *l'authentification* sur **Microsoft Entra MFA** et connectez-vous avec votre compte Azure. Dans VS Code, sélectionnez le type d'authentification **Microsoft Entra ID** lors de la création d'un profil de connexion.
- o **Authentification SQL** : Dans SSMS ou VS Code, saisissez la **connexion administrateur** serveur et le **mot de passe** que vous avez spécifiés lors de la création du serveur, avec *l'authentification* réglée sur **connexion SQL**.

Dans les deux cas, définissez le **nom du serveur** et la **base de données** sur **AdventureWorksLT**.

```
<your-server-name>.database.windows.net
```

2. Téléchargez le script de critique depuis [product-reviews-insert.sql](#) et enregistrez-le localement.
3. Ouvrez le fichier téléchargé et exécutez tout le script dans votre base de données **AdventureWorksLT**.
4. Vérifiez que la table a été créée et remplie en exécutant :

SQL

 Bien reçu

```
SELECT COUNT(*) AS TotalReviews FROM dbo.ProductReview;
GO
```

!  Vous devriez voir **140** rangées. Les avis couvrent les vélos, pneus, lampes, casques, gants, outils d'entretien, et plus encore, avec des notes allant de 1 à 5 étoiles.

Créer une base de données avec un champ d'identification et un modèle externe pour les embeddings

Configurez les identifiants et les références de modèle nécessaires pour appeler Azure OpenAI depuis T-SQL. Vous créez une seule certification en utilisant l'identité managée assignée par le système de votre Azure SQL Server. Cette certification est utilisée à la fois (pour les embeddings) et (pour les complétions de chat), et élimine le besoin de clés API. `CREATE EXTERNAL MODEL sp_invoke_external_rest_endpoint`

!  Passez cette section si vous avez déjà un identifiant avec un champ de données et un modèle d'embarquement externe configuré pour votre point de terminaison Azure OpenAI.

1. Pour créer une identifiante à portée de base de données en utilisant une identité gérée, ouvrez une nouvelle fenêtre de requête et exécutez le script suivant. Remplacez par le nom du terminau que vous avez indiqué dans l'onglet **OpenAI** (par exemple, si votre point de terminaison est , utilisez *proj-sqlailab-resource*). `<your-openai-endpoint>` `https://proj-sqlailab-resource.openai.azure.com/`

SQL

 Bien reçu



```
-- Create a database master key if one doesn't exist
IF NOT EXISTS (SELECT * FROM sys.symmetric_keys WHERE name = '##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<your strong password here>';
GO

-- Create a credential using Managed Identity
CREATE DATABASE SCOPED CREDENTIAL [https://<your-openai-endpoint>.openai.azure.com]
WITH IDENTITY = 'Managed Identity', SECRET =
'{"resourceid":"https://cognitiveservices.azure.com"}';
GO
```

!  Remplacez-le par un mot de passe fort.

Remplacez-le par le nom du point de terminaison depuis l'onglet **OpenAI** (par exemple, *proj-sqlailab-resource*).

<your strong password here>

<your-openai-endpoint>

⚠ **Important** : ne **modifiez pas** la valeur dans le SECRET. Il doit rester exactement . Il s'agit de l'audience OAuth fixe pour Azure Cognitive Services, pas de l'URL spécifique de votre endpoint. Le modifier provoquera des erreurs d'authentification. **resourceid**

https://cognitiveservices.azure.com

2. Créez maintenant une référence de modèle externe pour le modèle d'inclusion. Cette référence vous permet de l'utiliser directement en T-SQL. Remplacez-le par le nom de votre endpoint.

AI_GENERATE_EMBEDDINGS <your-openai-endpoint>

SQL

 Bien reçu

```
-- Create an external model reference for embeddings
CREATE EXTERNAL MODEL my_embedding_model
WITH (
    LOCATION = 'https://<your-openai-endpoint>.openai.azure.com/openai/deployments/text-embedding-3-small/embeddings?api-version=2024-10-21',
    API_FORMAT = 'Azure OpenAI',
    MODEL_TYPE = EMBEDDINGS,
    MODEL = 'text-embedding-3-small',
    CREDENTIAL = [https://<your-openai-endpoint>.openai.azure.com]
);
GO
```

!  Remplacez-le par le même nom de terminaison que vous avez utilisé dans l'identifiant. La valeur correspond à la version GA actuelle de l'API REST d'Azure OpenAI. Cette option est obligatoire et doit correspondre au nom du modèle. <your-openai-endpoint> api-version 2024-10-21 MODEL

Ajoutez une colonne vectorielle et générez des embeddings

Dans cette section, vous ajoutez une colonne vectorielle au tableau ProductReview, générez des insertions pour le texte de la critique, et créez un index vectoriel pour une recherche de similarité efficace.

!  Passez cette section si votre table contient déjà une colonne avec des embeddings générés. **dbo.ProductReview**

ReviewVector

1. Tout d'abord, ajoutez une colonne vectorielle au tableau pour stocker les embeddings de revue :

dbo.ProductReview



SQL

 Bien reçu

```
-- Add a vector column to store review text embeddings
ALTER TABLE dbo.ProductReview
ADD ReviewVector VECTOR(1536);
GO
```

!  Le type de données stocke un vecteur de dimension 1536, qui correspond à la sortie du modèle *text-embedding-3-small*. `VECTOR(1536)`

2. Générez des insertions pour chaque avis en combinant le nom du produit, le titre de la critique et le texte de l'avis. Le script traite les revues par lots de 30 avec un court délai entre les lots afin d'éviter les limites de débit de l'API :

SQL

 Bien reçu

[Prérequis](#)

[Provision an
Azure SQL
Database](#)

[Créez un projet
Foundry et
déployez des
modèles Azure
OpenAI](#)



[Configurer l'accès à l'identité gérée](#)

[Créer le tableau ProductReview](#)

Créer une base de données avec un champ d'identification et un modèle externe pour les embeddings

[Ajoutez une colonne vectorielle et générez des embeddings](#)

[Créer un index en texte intégral](#)

[Recherche avec prédicats en texte intégral](#)

[Recherche avec similarité vectorielle](#)

[Combinez la recherche en texte intégral et vectorielle avec la recherche hybride](#)

[Comparez les trois approches de recherche](#)

[Nettoyage](#)

```
-- Generate embeddings in batches to avoid API rate limits
DECLARE @batchSize INT = 30;
DECLARE @rowsUpdated INT = 1;
DECLARE @retryCount INT;
DECLARE @maxRetries INT = 3;

WHILE @rowsUpdated > 0
BEGIN
    SET @retryCount = 0;

    RETRY:
    BEGIN TRY
        UPDATE TOP (@batchSize) r
        SET r.ReviewVector = AI_GENERATE_EMBEDDINGS(
            p.Name + ' - ' + r.ReviewTitle + ': ' + r.ReviewText
            USE MODEL my_embedding_model)
        FROM dbo.ProductReview r
        INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
        WHERE r.ReviewVector IS NULL;

        SET @rowsUpdated = @@ROWCOUNT;

        -- Brief pause between batches to respect API rate limits
        IF @rowsUpdated > 0
            WAITFOR DELAY '00:00:02';
    END TRY
    BEGIN CATCH
        SET @retryCount += 1;
        IF @retryCount <= @maxRetries
        BEGIN
            PRINT 'Rate limited. Retrying in 5 seconds... (Attempt '
                + CAST(@retryCount AS NVARCHAR(10)) + ' of '
                + CAST(@maxRetries AS NVARCHAR(10)) + ')';
            WAITFOR DELAY '00:00:05';
            GOTO RETRY;
        END
        ELSE
            THROW;
    END CATCH
END
GO
```

!  This step may take a couple of minutes. The script processes 30 reviews per batch with a 2-second pause between batches. If the API returns a rate-limit error, it retries up to three times with a 5-second wait. The product name, review title, and review text are embedded together so that vector search can match on both the product being reviewed and the customer's experience.

3. Verify that the embeddings were generated:

Sql

 Copy



```
-- Check how many reviews have embeddings
SELECT
    COUNT(*) AS TotalReviews,
    COUNT(ReviewVector) AS ReviewsWithEmbeddings
FROM dbo.ProductReview;
GO
```

4. Enable preview features and create a vector index on the column for efficient approximate nearest neighbor (ANN) search:

Sql

 Copy

```
-- Enable preview features required for vector indexes
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;
GO

-- Allow the table to remain writable after vector index creation
ALTER DATABASE SCOPED CONFIGURATION SET ALLOW_STALE_VECTOR_INDEX = ON;
GO

-- Create a DiskANN vector index for fast approximate nearest neighbor search
CREATE VECTOR INDEX IX_Review_ReviewVector
ON dbo.ProductReview(ReviewVector)
WITH (METRIC = 'cosine', TYPE = 'DISKANN');
GO
```

  creates a DiskANN-based approximate nearest neighbor (ANN) index, which is fundamentally different from a regular nonclustered index. DiskANN builds a graph structure that navigates through vectors to find close matches efficiently. The setting keeps the table writable. Without it, the table becomes read-only when a vector index exists.

CREATE VECTOR INDEX **ALLOW_STALE_VECTOR_INDEX**

Create a full-text index

Full-text search requires a full-text index on the text columns you want to search. In this section, you create a full-text catalog and index on the and columns of the table. **ReviewTitle** **ReviewText** **ProductReview**

1. Create a full-text catalog and a full-text index:

Sql

 Copy



```
-- Create a full-text catalog
CREATE FULLTEXT CATALOG ProductReviewCatalog AS DEFAULT;
GO

-- Create a full-text index on ReviewTitle and ReviewText
CREATE FULLTEXT INDEX ON dbo.ProductReview
(
    ReviewTitle LANGUAGE 1033,
    ReviewText LANGUAGE 1033
)
KEY INDEX PK_ProductReview
ON ProductReviewCatalog
WITH (CHANGE_TRACKING AUTO);
GO
```

! 🖨 The must reference the unique index on the table's primary key. specifies English, which enables inflectional matching (for example, "ride" matching "riding"). keeps the index updated as data changes. **KEY INDEX**

LANGUAGE 1033 **CHANGE_TRACKING AUTO**

2. Verify the full-text index was created and is populated:

Sql

📄 Copy

```
-- Check full-text index status
SELECT
    OBJECTPROPERTY(OBJECT_ID('dbo.ProductReview'), 'TableFullTextPopulateStatus') AS
PopulateStatus,
    OBJECTPROPERTY(OBJECT_ID('dbo.ProductReview'), 'TableHasActiveFulltextIndex') AS
HasActiveIndex;
GO
```

! 🖨 A of **0** means the full-text index is fully populated and ready for queries. A value of **1** means population is still in progress. should be **1**. **PopulateStatus** **HasActiveIndex**

Search with full-text predicates

Full-text search uses predicates like and to query the full-text index. looks for exact words or phrases. matches word forms and inflections automatically. **CONTAINS** **FREETEXT** **CONTAINS** **FREETEXT**

1. Use to search for reviews mentioning a specific word: **CONTAINS**

Sql

📄 Copy



```
-- Find reviews that contain the word "puncture"
SELECT
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    p.Name AS ProductName
FROM dbo.ProductReview r
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
WHERE CONTAINS(r.ReviewText, 'puncture');
GO
```

! 🗨️ searches the full-text index for the exact word "puncture." It returns only reviews where that specific word appears. **CONTAINS**

2. Use to search for a phrase. automatically expands search terms to include inflectional forms: **FREETEXT**

FREETEXT

Sql Copy

```
-- Find reviews about gloves and warmth
SELECT TOP 10
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    p.Name AS ProductName
FROM dbo.ProductReview r
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
WHERE FREETEXT(r.ReviewText, 'warm gloves for cold winter commuting');
GO
```

! 🗨️ handles inflections, so "warm" can match "warmth" or "warming," and "commuting" can match "commute" or "commuter." It also drops stopwords like "for" and "the." Compare this to **CONTAINS**, which would require you to specify each word form explicitly. Use with **TOP** since broad phrases can match many rows. **FREETEXT** **CONTAINS** **TOP**

FREETEXT

FREETEXT is more flexible, but it might return less relevant results if the search phrase is too broad. gives you more control but requires more precise queries. **CONTAINS**

3. Use to get ranked results with relevance scores: **CONTAINSTABLE**

Sql Copy



```
-- Search with ranking scores
SELECT TOP 10
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    p.Name AS ProductName,
    ft.[RANK] AS FullTextRank
FROM dbo.ProductReview r
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
INNER JOIN CONTAINSTABLE(dbo.ProductReview, (ReviewTitle, ReviewText),
    'FORMSOF(INFLECTIONAL, mountain) AND FORMSOF(INFLECTIONAL, trail)') AS ft
    ON r.ReviewID = ft.[KEY]
ORDER BY ft.[RANK] DESC;
GO
```

! 📖 returns a table with a column (matching the primary key) and a column indicating how well each row matches. matches "mountain," "mountains," and other inflected forms. The operator requires both terms to appear.

CONTAINSTABLE KEY RANK FORMSOF(INFLECTIONAL, mountain) AND

4. Use a prefix search to find reviews where words start with specific characters:

Sql

Copy

```
-- Find reviews with words starting with "comfort"
SELECT
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    p.Name AS ProductName
FROM dbo.ProductReview r
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
WHERE CONTAINS(r.ReviewText, '"comfort*");
GO
```

! 📖 The prefix search matches "comfort," "comfortable," "comfortably," and any other word beginning with "comfort." This is useful when you want to capture variations of a root word without listing every form.

"comfort"

Search with vector similarity

Vector search finds reviews based on the semantic meaning of text, not just keyword matches. A question about "keeping drinks cold on a ride" can find reviews about water bottles even if those exact words don't appear.

1. Use for exact nearest neighbor search. This option calculates the cosine distance between the query embedding and every review: VECTOR_DISTANCE

Sql

Copy



```
-- Exact vector search using VECTOR_DISTANCE
DECLARE @searchText NVARCHAR(1000) = 'comfortable seat for long distance touring';
DECLARE @searchVector VECTOR(1536);

SELECT @searchVector = AI_GENERATE_EMBEDDINGS(@searchText USE MODEL my_embedding_model);

SELECT TOP 5
    p.Name AS ProductName,
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    VECTOR_DISTANCE('cosine', @searchVector, r.ReviewVector) AS Distance
FROM dbo.ProductReview r
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
ORDER BY Distance;
GO
```

 calculates the cosine distance between two vectors. Lower values mean higher similarity. This function scans every row in the table, which works well for smaller datasets. Notice that the results include reviews about touring bikes and saddle comfort even if they do not contain the exact words “comfortable seat.” **VECTOR_DISTANCE**

2. Use with the DiskANN index for approximate nearest neighbor (ANN) search. This index is optimized for large datasets and provides faster results: **VECTOR_SEARCH**

Sql

 Copy

```
-- Approximate vector search using VECTOR_SEARCH with DiskANN index
DECLARE @searchText NVARCHAR(1000) = 'something to keep me visible when riding at night';
DECLARE @searchVector VECTOR(1536);

SELECT @searchVector = AI_GENERATE_EMBEDDINGS(@searchText USE MODEL my_embedding_model);

SELECT
    p.Name AS ProductName,
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    pc.Name AS Category,
    vs.distance AS Distance
FROM VECTOR_SEARCH(
    TABLE = dbo.ProductReview AS r,
    COLUMN = ReviewVector,
    SIMILAR_TO = @searchVector,
    METRIC = 'cosine',
    TOP_N = 5
) AS vs
INNER JOIN SalesLT.Product p
    ON r.ProductID = p.ProductID
INNER JOIN SalesLT.ProductCategory pc
    ON p.ProductCategoryID = pc.ProductCategoryID
ORDER BY vs.distance;
GO
```



! 📄 uses the DiskANN index to find approximate nearest neighbors without scanning every row. The query describes a concept ("visible when riding at night") rather than specific keywords. Vector search finds reviews about lights, reflective gear, and visibility even when those words do not appear in the search text. `VECTOR_SEARCH`

3. Check vector metadata using : `VECTORPROPERTY`

Sql

Copy

```
-- Inspect vector metadata
SELECT TOP 1
    VECTORPROPERTY(ReviewVector, 'Dimensions') AS VectorDimensions,
    VECTORPROPERTY(ReviewVector, 'BaseType') AS VectorBaseType
FROM dbo.ProductReview
WHERE ReviewVector IS NOT NULL;
GO
```

! 📄 returns metadata about a vector column. This is useful for validating that your vectors have the expected number of dimensions and confirming the data type. `VECTORPROPERTY`

Combine full-text and vector search with hybrid search

Full-text search excels at finding exact keywords but misses documents that express the same idea differently. Vector search captures semantic meaning but might miss important terms the user specified. Hybrid search runs both approaches and merges the results using Reciprocal Rank Fusion (RRF).

RRF combines ranked results from different sources by using rank positions instead of raw scores. The formula converts ranks into scores, where is a smoothing constant (typically 60). Items appearing in both result sets get higher combined scores, pushing the most broadly relevant results to the top. $\frac{1}{(k + \text{rank})}$ `k`

1. Run the following hybrid search query that combines full-text and vector search using RRF:

Sql

Copy



```
DECLARE @searchText NVARCHAR(1000) = 'durable tires that resist punctures on rough
terrain';
DECLARE @searchVector VECTOR(1536);
DECLARE @topN INT = 50;
DECLARE @rrfK INT = 60;

-- Generate embedding for the search phrase
SELECT @searchVector = AI_GENERATE_EMBEDDINGS(@searchText USE MODEL my_embedding_model);

-- Run hybrid search with RRF
WITH keyword_search AS (
    SELECT TOP(@topN)
        r.ReviewID,
        RANK() OVER (ORDER BY ft.[RANK] DESC) AS keyword_rank
    FROM dbo.ProductReview r
    INNER JOIN FREETEXTTABLE(dbo.ProductReview, (ReviewTitle, ReviewText), @searchText) AS
ft
        ON r.ReviewID = ft.[KEY]
),
vector_search AS (
    SELECT TOP(@topN)
        ReviewID,
        RANK() OVER (ORDER BY distance) AS vector_rank
    FROM (
        SELECT
            r.ReviewID,
            vs.distance
        FROM VECTOR_SEARCH(
            TABLE = dbo.ProductReview AS r,
            COLUMN = ReviewVector,
            SIMILAR_TO = @searchVector,
            METRIC = 'cosine',
            TOP_N = 50
        ) AS vs
    ) AS similar_reviews
),
combined AS (
    SELECT
        COALESCE(ks.ReviewID, vs.ReviewID) AS ReviewID,
        ks.keyword_rank,
        vs.vector_rank,
        COALESCE(1.0 / (@rrfK + ks.keyword_rank), 0.0) +
        COALESCE(1.0 / (@rrfK + vs.vector_rank), 0.0) AS rrf_score
    FROM keyword_search ks
    FULL OUTER JOIN vector_search vs ON ks.ReviewID = vs.ReviewID
)
SELECT TOP 10
    p.Name AS ProductName,
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    c.keyword_rank,
    c.vector_rank,
    c.rrf_score
FROM combined c
INNER JOIN dbo.ProductReview r ON c.ReviewID = r.ReviewID
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
```



```
ORDER BY c.rrf_score DESC;
GO
```

 This query runs full-text and vector search in parallel using CTEs. The CTE uses to rank reviews by BM25 relevance. The CTE uses to rank reviews by embedding similarity. The CTE joins both result sets with a and calculates the RRF score. Reviews that appear in both lists get higher combined scores. The final returns the top 10 results ordered by RRF score, showing which reviews performed well across both search methods.

```
FREETEXTTABLE vector_search VECTOR_SEARCH combined FULL OUTER JOIN SELECT
```

2. Examine the output columns. Rows where both and have values were found by both search methods and tend to have the highest RRF scores. Rows with a in one rank column were found by only one method.

```
keyword_rank vector_rank NULL
```

Compare the three search approaches

To understand the strengths of each approach, run the same question through all three search methods and compare the results.

1. Run a full-text search for a comfortable family bike:

Sql

 Copy

```
-- Full-text search only
SELECT TOP 5
    p.Name AS ProductName,
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    ft.[RANK] AS FullTextRank
FROM dbo.ProductReview r
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
INNER JOIN FREETEXTTABLE(dbo.ProductReview, (ReviewTitle, ReviewText),
    'comfortable bike for long weekend rides with the family') AS ft
    ON r.ReviewID = ft.[KEY]
ORDER BY ft.[RANK] DESC;
GO
```

2. Run a vector search with the same question:

Sql

 Copy



```
-- Vector search only
DECLARE @searchText NVARCHAR(1000) = 'comfortable bike for long weekend rides with the
family';
DECLARE @searchVector VECTOR(1536);

SELECT @searchVector = AI_GENERATE_EMBEDDINGS(@searchText USE MODEL my_embedding_model);

SELECT
    p.Name AS ProductName,
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    vs.distance AS Distance
FROM VECTOR_SEARCH(
    TABLE = dbo.ProductReview AS r,
    COLUMN = ReviewVector,
    SIMILAR_TO = @searchVector,
    METRIC = 'cosine',
    TOP_N = 5
) AS vs
INNER JOIN SalesLT.Product p
    ON r.ProductID = p.ProductID
ORDER BY vs.distance;
GO
```

3. Run the hybrid search with the same question:

Sql

 Copy



```
-- Hybrid search with RRF
DECLARE @searchText NVARCHAR(1000) = 'comfortable bike for long weekend rides with the
family';
DECLARE @searchVector VECTOR(1536);
DECLARE @topN INT = 50;
DECLARE @rrfK INT = 60;

SELECT @searchVector = AI_GENERATE_EMBEDDINGS(@searchText USE MODEL my_embedding_model);

WITH keyword_search AS (
    SELECT TOP(@topN)
        r.ReviewID,
        RANK() OVER (ORDER BY ft.[RANK] DESC) AS keyword_rank
    FROM dbo.ProductReview r
    INNER JOIN FREETEXTTABLE(dbo.ProductReview, (ReviewTitle, ReviewText), @searchText) AS
ft
        ON r.ReviewID = ft.[KEY]
),
vector_search AS (
    SELECT TOP(@topN)
        ReviewID,
        RANK() OVER (ORDER BY distance) AS vector_rank
    FROM (
        SELECT
            r.ReviewID,
            vs.distance
        FROM VECTOR_SEARCH(
            TABLE = dbo.ProductReview AS r,
            COLUMN = ReviewVector,
            SIMILAR_TO = @searchVector,
            METRIC = 'cosine',
            TOP_N = 50
        ) AS vs
    ) AS similar_reviews
),
combined AS (
    SELECT
        COALESCE(ks.ReviewID, vs.ReviewID) AS ReviewID,
        ks.keyword_rank,
        vs.vector_rank,
        COALESCE(1.0 / (@rrfK + ks.keyword_rank), 0.0) +
        COALESCE(1.0 / (@rrfK + vs.vector_rank), 0.0) AS rrf_score
    FROM keyword_search ks
    FULL OUTER JOIN vector_search vs ON ks.ReviewID = vs.ReviewID
)
SELECT TOP 5
    p.Name AS ProductName,
    r.ReviewTitle,
    r.ReviewText,
    r.Rating,
    c.keyword_rank,
    c.vector_rank,
    c.rrf_score
FROM combined c
INNER JOIN dbo.ProductReview r ON c.ReviewID = r.ReviewID
INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
```



```
ORDER BY c.rrf_score DESC;  
GO
```

!  Compare the three result sets side by side. Full-text search returns reviews containing words like “comfortable,” “weekend,” and “family.” Vector search returns reviews about recreational bikes, relaxed geometry, and leisure riding that might use different wording. Hybrid search combines both, giving higher scores to reviews that appear in both result sets. This comparison illustrates when each approach works best: full-text for exact keyword matches, vector for semantic similarity, and hybrid when you want the best of both.

Cleanup

If you aren't using the Azure SQL Database or the Azure OpenAI resources for any other purpose, you can clean up the resources you created in this exercise.

!  These resources are used on labs 9, 10, and 11.

If you provisioned a new resource group for this lab, you can simply delete the entire resource group to remove all resources at once. If you used an existing resource group, delete the Azure SQL Database and Azure OpenAI resource individually.

1. In the Azure portal, navigate to your resource group.
2. Select **Delete resource group** and confirm deletion by typing the resource group name.
3. Select **Delete** to remove all resources created in this lab.

You successfully completed this exercise.

In this exercise, you implemented and compared three search approaches in Azure SQL Database: full-text search using a full-text index with keyword predicates and inflectional patterns, vector search using exact and approximate nearest neighbor queries with a DiskANN index, and hybrid search combining both methods with Reciprocal Rank Fusion to merge keyword and semantic results. You compared all three approaches on the same query to understand the strengths of each.